DOKUMEN SPESIFIKASI TEKNIS & STANDAR OPERASIONAL SISTEM

Rekayasa Arsitektur RESTful API, Serialisasi Resource, Validasi Semantik, dan Deployment Cloud — Proyek Showroom Mobil BTNG 2026

Identitas Proyek: Tugas Akhir BTNG Divisi Website 2026	**Klasifikasi:** Backend RESTful API Specification
Nama Pengembang: Arkaan Taqiyyuddin	**Lembaga:** UKM Dian Nuswantoro Computer Club (DNCC)
Tech Stack Inti: Laravel 12.x | PHP 8.5/8.2+ | MySQL 8.0	**API Standard:** OpenAPI 3.1 (Scramble) | Envelope Pattern
GitHub: github.com/Arrsiixx7/BTNG-Backend-ArkaanTaqiyyuddin	**Cloud Platform:** Clever Cloud PaaS (FrankenPHP/Apache)

1. ARSITEKTUR SISTEM & PRINSIP DESAIN PERANGKAT LUNAK

Sistem dibangun di atas fondasi arsitektur Model-View-Controller (MVC) yang diadaptasi secara khusus untuk layanan berbasis headless API (tanpa View template Blade). Lapisan presentasi digantikan sepenuhnya oleh **Transformation Layer (API Resource)** yang menjamin isolasi ketat antara struktur internal skema database dengan antarmuka data publik yang dikonsumsi oleh klien Frontend.

Prinsip Arsitektural Utama:
• **Stateless Communication:** Setiap request HTTP membawa seluruh konteks yang diperlukan untuk dieksekusi tanpa bergantung pada session state server.
• **HTTP Semantic Compliance:** Penggunaan kata kerja HTTP standar (GET, POST, PUT, DELETE) yang konsisten terhadap sifat idempoten dan keamanan operasinya.
• **Envelope Serialization:** Semua respons sistem dienkapsulasi seragam ke dalam struktur amplop berkunci message dan data (atau errors saat validasi gagal) guna menyederhanakan parsing pada consumer client.
• **Defensive Validation Layer:** Validasi data masuk diterapkan di baris terdepan Controller sebelum menyentuh lapisan ORM, mencegah anomali tipe dan injeksi data.

2. MATRIKS STANDAR EVALUASI & DISTRIBUSI BOBOT PENILAIAN

No.	Komponen Evaluasi	Parameter Teknis & Bukti Implementasi	Bobot
1	**Fungsionalitas CRUD API**	Implementasi 5 endpoint semantik penuh (Index, Store, Show, Update, Destroy). Transaksi basis data berjalan tanpa anomali menggunakan Eloquent ORM. Metode HTTP GET bersifat safe/idempotent, PUT idempotent, dan DELETE idempotent.	**30%**
2	**Dokumentasi Interaktif (Scramble)**	Dokumentasi OpenAPI 3.1 terpasang via Dedoc Scramble di /docs/api. Skema request body, parameter query, dan respon JSON terdeteksi secara otomatis, dilengkapi fungsionalitas pengujian langsung (*Try It Out*) dan Gate otorisasi publik.	**20%**
3	**Validasi Data & API Resource**	Integritas data dijamin melalui $request->validate() dengan pesan penolakan yang presisi. Respon seragam dibungkus oleh JsonResource (amplop message dan data), disertai status kode HTTP semantik (200, 201, 404, 422).	**20%**
4	**Deployment Live ke Cloud**	Layanan aktif secara publik di infrastruktur PaaS Clever Cloud dengan konektivitas database cloud eksternal. Akses bebas hambatan CORS (Cross-Origin Resource Sharing) dari domain frontend manapun.	**15%**
5	**Arsitektur, Seeder & Kerapian Kode**	Penerapan Clean Code, Mass Assignment Protection via $fillable, Seeder awal minimal 5-8 data dummy dengan pencegahan duplikasi, ketersediaan .env.example bersih, dan dokumentasi README komprehensif.	**15%**

3. SPESIFIKASI SKEMA BASIS DATA & MODEL ELOQUENT

Tabel utama dinamakan mobils sesuai konvensi penamaan jamak Laravel. Didesain dengan engine InnoDB untuk menjamin kepatuhan integritas ACID dan performa pengindeksan yang optimal.

Kolom Database	Tipe Data SQL	Constraint & Nullable	Aturan Validasi Request	Deskripsi & Peran Entitas
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	N/A (Generated by DB)	Pengenal unik record entitas kendaraan.
nama	VARCHAR(255)	NOT NULL	required|string|max:255	Model dan varian mobil (contoh: Avanza 1.5 G CVT).
brand	VARCHAR(100)	NOT NULL, INDEXED	required|string|max:100	Nama manufaktur pabrikan (contoh: Toyota, Honda).
harga	BIGINT UNSIGNED	NOT NULL	required|numeric|min:0	Nilai harga nominal dalam satuan mata uang Rupiah.
tahun	INT	NOT NULL	required|integer|min:1900|max:Y+1	Tahun perakitan resmi unit kendaraan bermotor.
transmisi	VARCHAR(50)	NOT NULL	required|string|max:50	Sistem penggerak transmisi (Manual, Matic, CVT).
bahan_bakar	VARCHAR(50)	NOT NULL	required|string|max:50	Sumber energi daya (Bensin, Diesel, Hybrid, Listrik).
status	VARCHAR(20)	NOT NULL, DEFAULT 'tersedia'	required|in:tersedia,sold	Status ketersediaan logistik unit showroom.
foto	TEXT	NOT NULL	required|string|url	URL aset gambar kendaraan resolusi tinggi.
created_at	TIMESTAMP	NULLABLE	N/A (Managed by Eloquent)	Timestamp presisi pembuatan record pertama kali.
updated_at	TIMESTAMP	NULLABLE	N/A (Managed by Eloquent)	Timestamp presisi pembaruan record terakhir.

4. KONTRAK ENDPOINT API & ANALISIS METODE HTTP SEMANTIK

HTTP Verb	Uniform Resource Identifier (URI)	Karakteristik Operasi	Status Sukses	Status Kegagalan
GET	/api/mobil	Read Collection: Mengambil kumpulan record dengan dukungan filter multi-atribut dan pencarian teks.	**200 OK**	500 Server Error
GET	/api/mobil/{id}	Read Single: Mengambil detail spesifik satu record mobil berdasarkan parameter pengenal ID.	**200 OK**	404 Not Found
POST	/api/mobil	Create: Menerima payload JSON untuk divalidasi dan dicatat sebagai baris data baru dalam tabel.	**201 Created**	422 Unprocessable Content
PUT / PATCH	/api/mobil/{id}	Update: Melakukan pembaruan menyeluruh atau mutasi parsial terhadap atribut entitas yang tersimpan.	**200 OK**	404 Not Found 422 Unprocessable
DELETE	/api/mobil/{id}	Delete: Mengeliminasi data unit kendaraan dari basis data secara permanen.	**200 OK**	404 Not Found
GET	/docs/api	API Documentation: Halaman grafis interaktif OpenAPI 3.1 yang dirender oleh Stoplight Elements.	**200 OK**	403 Forbidden (jika Gate tertutup)

5. STANDARISASI SERIALISASI PAYLOAD JSON (ENVELOPE PATTERN)

Mengacu pada standar rancang bangun API kelas industri, respons sistem tidak boleh memulangkan array mentah tanpa konteks. Format amplop (*envelope*) menyediakan konsistensi struktur baik untuk pemrosesan sukses maupun kegagalan validasi.

Pola 1: Respons Pengambilan Koleksi Data (200 OK)	Pola 2: Respons Penolakan Validasi Input (422 Unprocessable)
`{ "message": "Daftar mobil berhasil diambil", "data": [{ "id": 1, "nama": "Avanza 1.5 G CVT", "brand": "Toyota", "harga": 275000000, "tahun": 2024, "transmisi": "CVT", "bahan_bakar": "Bensin", "status": "tersedia", "foto": "https://images.unsplash.com/...", "created_at": "2026-10-27T00:53:00Z" }] }`	`{ "message": "Kolom nama mobil wajib diisi. (and 1 more error)", "errors": { "nama": [ "Kolom nama mobil wajib diisi." ], "harga": [ "Kolom harga harus berupa angka." ] } }`

6. PANDUAN PELAKSANAAN PROSEDUR REKAYASA SISTEM (10 TAHAPAN UTAMA)

Berikut merupakan dokumentasi operasional terperinci mengenai tahapan konstruksi kode, konfigurasi infrastruktur, dan instruksi eksekusi pada setiap layer aplikasi:

Tahap 1: Inisialisasi Proyek, Konfigurasi Repositori Git & Audit Keamanan

Tujuan & Analisis: Membangun ruang kerja Laravel 12 yang terisolasi dan mandiri, serta menerapkan perlindungan kredensial ketat agar rahasia infrastruktur (database secret, application key) tidak ter-expose ke repositori publik.

Aktivitas Rekayasa:

1. Eksekusi pembuatan repositori lokal Git pada root folder proyek: git init -b main.
2. Konfigurasi file .gitignore: Wajib mengecualikan .env, .env.backup, vendor/, storage/*.key, dan database/*.sqlite.
3. Pembuatan file .env.example yang telah disanitasi: Berisi daftar seluruh parameter variabel lingkungan yang dibutuhkan sistem dengan nilai kosong atau nilai contoh pengujian, sebagai acuan setup bagi penguji dan tim lain.

Tahap 2: Konstruksi Skema Database Menggunakan Laravel Migration

Tujuan & Analisis: Menyusun migrasi skema tabel secara deklaratif dan version-controlled, menghindari modifikasi DDL manual.

Aktivitas Rekayasa:

1. Eksekusi artisan generator: php artisan make:model Mobil -m.
2. Implementasi method up() pada file database/migrations/xxxx_create_mobils_table.php:
   - $table->id(): Auto-increment primary key 64-bit.
   - $table->string('nama'), $table->string('brand'): Identifikasi unit dan merk.
   - $table->unsignedBigInteger('harga'): Kapasitas angka finansial non-negatif hingga 18 triliun.
   - $table->integer('tahun'): Tahun rakit kendaraan.
   - $table->string('transmisi'), $table->string('bahan_bakar'): Spesifikasi mekanikal.
   - $table->string('status')->default('tersedia'): Flag ketersediaan inventori.
   - $table->text('foto'): Penyimpanan teks panjang untuk link URL gambar eksternal.
   - $table->timestamps(): Tracking kronologis created_at dan updated_at.
3. Eksekusi migrasi: php artisan migrate.

Tahap 3: Implementasi Model Eloquent & Perlindungan Mass Assignment

Tujuan & Analisis: Membangun representasi entitas data berbasis Active Record Pattern serta menutup kerentanan Mass Assignment.

Aktivitas Rekayasa:

1. Berkas: app/Models/Mobil.php.
2. Konfigurasi protected $fillable: Mendaftarkan secara eksplisit atribut-atribut yang boleh diisi via array payload: ['nama', 'brand', 'harga', 'tahun', 'transmisi', 'bahan_bakar', 'status', 'foto']. Properti ini mencegah manipulasi kolom sistem secara ilegal oleh pengguna luar.
3. Konfigurasi casts(): Menetapkan casting tipe data native PHP untuk harga (integer) dan tahun (integer).

Tahap 4: Database Seeding & Pencegahan Duplikasi Data

Tujuan & Analisis: Menjamin ketersediaan data awal yang representatif (minimal 5-8 variasi unit) untuk keperluan pengujian dan review.

Aktivitas Rekayasa:

1. Generator: php artisan make:seeder MobilSeeder.
2. Strategi Idempoten: Menggunakan method Mobil::firstOrCreate(['nama' => ...], [...]) di dalam MobilSeeder.php. Pendekatan ini menjamin bahwa seeder aman dijalankan berulang kali (pada deployment hook) tanpa mengakibatkan duplikasi data entitas.
3. Registrasi: Mendaftarkan MobilSeeder::class ke dalam runner utama database/seeders/DatabaseSeeder.php.
4. Eksekusi penanaman data: php artisan db:seed.

Tahap 5: Enkapsulasi Respons Melalui API Resource (Transformation Layer)

Tujuan & Analisis: Membentuk layer isolasi antara model database dengan output JSON, mencegah kebocoran informasi internal.

Aktivitas Rekayasa:

1. Generator: php artisan make:resource MobilResource.
2. Berkas: app/Http/Resources/MobilResource.php.
3. Implementasi toArray(Request $request): Mengembalikan array terstruktur dengan pemformatan tipe eksplisit serta standardisasi tanggal menggunakan format ISO 8601 (toISOString()).

Tahap 6: Konstruksi Controller & Logika Bisnis 5 Endpoint CRUD

Tujuan & Analisis: Mengelola aliran kendali permintaan masuk, validasi parameter, operasi persistensi Eloquent, dan respons status code.

Berkas Implementasi: app/Http/Controllers/Api/MobilController.php.

Detail Bedah Fungsi:

• **index(Request $request):** Menginisialisasi Eloquent Query Builder. Memeriksa parameter query search (menerapkan operator LIKE pada kolom nama dan brand). Menangani filtering spesifik berdasarkan brand, status, bahan_bakar, dan transmisi. Data diurutkan secara terbalik (latest('id')) dan dikembalikan via MobilResource::collection() dengan status 200 OK.

• **store(Request $request):** Memvalidasi seluruh atribut masukan melalui $request->validate() dengan pesan penolakan kustom berbahasa Indonesia. Atribut status dinormalisasi menjadi huruf kecil. Entitas baru dicatat via Mobil::create() dan dikembalikan dengan status 201 Created.

• **show(string|int $id):** Mengambil satu entitas melalui Mobil::find($id). Menerapkan pemeriksaan defensif: jika instans bernilai null, mengembalikan respons JSON 404 Not Found. Jika ditemukan, mengembalikan model dalam bungkus MobilResource berstatus 200 OK.

• **update(Request $request, string|int $id):** Mencari data target (404 jika absen). Menerapkan validasi bersyarat menggunakan prefix sometimes|required, mengizinkan pembaruan penuh maupun mutasi parsial. Memperbarui record melalui $mobil->update() dan mengembalikan entitas segar ($mobil->fresh()) berstatus 200 OK.

• **destroy(string|int $id):** Mencari target entitas (404 jika absen), mengeksekusi $mobil->delete(), dan mengembalikan konfirmasi pesan penghapusan sukses berstatus 200 OK.

Tahap 7: Deklarasi Routing API & Konfigurasi Kebijakan Akses CORS

Tujuan & Analisis: Memetakan URI endpoint secara bersih dan mengizinkan komunikasi lintas domain (Cross-Origin) dari Frontend.

Aktivitas Rekayasa:

1. Berkas routes/api.php: Dideklarasikan menggunakan Route::apiResource('mobil', MobilController::class); yang secara otomatis menghasilkan 5 rute RESTful tanpa menyertakan rute create/edit (khas aplikasi web bertemplate).
2. Publikasi dan konfigurasi file CORS: php artisan config:publish cors.
3. Berkas config/cors.php: Memastikan 'paths' => ['api/*', ...] dan 'allowed_origins' => ['*'] terpasang agar browser mengizinkan aplikasi Frontend (seperti Veloce Auto) mengeksekusi Native Fetch API tanpa penolakan CORS.

Tahap 8: Integrasi Engine Dokumentasi Interaktif Dedoc Scramble

Tujuan & Analisis: Menyediakan dokumentasi API interaktif berstandar OpenAPI 3.1 tanpa overhead pemeliharaan manual.

Aktivitas Rekayasa:

1. Instalasi dependensi: composer require dedoc/scramble.
2. Konfigurasi Otorisasi Akses: Scramble secara default menolak akses non-lokal (production). Agar dokumentasi dapat dibuka secara publik di Clever Cloud, didaftarkan Gate otorisasi pada app/Providers/AppServiceProvider.php:
   Gate::define('viewApiDocs', fn($user = null) => true);
3. Publikasi konfigurasi: php artisan vendor:publish --tag=scramble-config.
4. Kustomisasi informasi API pada config/scramble.php meliputi Title, Version (1.0.0), dan deskripsi proyek.
5. Output spesifikasi OpenAPI tersedia secara native pada URL /docs/api (UI) dan /docs/api.json (Raw Schema).

Tahap 9: Pengujian Otomatis Menyeluruh (Automated Feature Testing)

Tujuan & Analisis: Memvalidasi keandalan seluruh endpoint, respon skema, validasi gagal, dan penanganan error melalui automated testing suite.

Aktivitas Rekayasa:

1. Pembuatan berkas pengujian: tests/Feature/MobilApiTest.php.
2. Penggunaan trait RefreshDatabase guna memastikan setiap skenario pengujian berjalan pada sandbox database yang bersih.
3. Rincian Skenario Pengujian (13 Tests, 75 Assertions):
   - Pengambilan seluruh koleksi dan verifikasi struktur schema JSON (200 OK).
   - Pengujian efektivitas query parameter filter brand, status, dan search.
   - Pengambilan detail entitas valid (200 OK) dan entitas non-eksisten (404 Not Found).
   - Penambahan data baru sukses (201 Created) dan verifikasi persistensi assertDatabaseHas.
   - Pengujian kegagalan validasi payload kosong/invalid (422 Unprocessable Content).
   - Pengujian pembaruan mutasi parsial (200 OK) dan penanganan ID salah pada update (404 Not Found).
   - Pengujian penghapusan entitas (200 OK) dan verifikasi hilangnya data assertDatabaseMissing.
   - Pengujian aksesibilitas endpoint dokumentasi Scramble di /docs/api (200 OK).
4. Eksekusi pengujian: php artisan test. Hasil pengujian menunjukkan status 100% PASS.

Tahap 10: Konfigurasi Deployment Infrastruktur Cloud (Clever Cloud)

Tujuan & Analisis: Men-deploy aplikasi ke PaaS Clever Cloud dengan konfigurasi webroot, database add-on, dan migration hook otomatis.

Aktivitas Rekayasa:

1. Routing Webroot Document: Clever Cloud mengeksekusi Apache/PHP dari direktori root. Agar terarah ke direktori publik Laravel, dibuat berkas konfigurasi .clever.cloud/php.json dan clevercloud/php.json:
   {"deploy": {"webroot": "/public"}}.
2. Penyesuaian Driver Database Cloud (config/database.php): Menambahkan fallback cerdas terhadap environment variables add-on MySQL Clever Cloud:
   'host' => env('MYSQL_ADDON_HOST', env('DB_HOST', '127.0.0.1'))
   'database' => env('MYSQL_ADDON_DB', env('DB_DATABASE', 'laravel'))
   'username' => env('MYSQL_ADDON_USER', env('DB_USERNAME', 'root'))
   'password' => env('MYSQL_ADDON_PASSWORD', env('DB_PASSWORD', '')).
3. Konfigurasi Environment Variables di Console Clever Cloud:
   - APP_ENV=production | APP_DEBUG=false
   - APP_KEY: Nilai enkripsi dari php artisan key:generate --show
   - CC_WEBROOT=/public
   - CC_POST_BUILD_HOOK=php artisan migrate --force && php artisan db:seed --force
4. Deployment Git: git push clever main:master.

7. CHECKLIST AUDIT KUALITAS & PROTOKOL PENGUMPULAN

Item Pemeriksaan	Standar Kriteria Kepatuhan	Status Hasil Audit
Integritas CRUD	5 endpoint semantik berfungsi 100% pada pengujian lokal maupun live cloud deployment.	**TERVERIFIKASI**
Scramble OpenAPI	UI /docs/api dapat diakses publik, schema request/response akurat, fitur 'Try it out' aktif.	**TERVERIFIKASI**
Keamanan .env	File .env lokal diabaikan Git. File .env.example bersih tersedia di repositori.	**TERVERIFIKASI**
Automated Tests	13 skenario tes Feature dan Unit (75 assertions) berhasil dieksekusi dengan status 100% lolos.	**TERVERIFIKASI**
CORS Compliance	Header CORS mengizinkan origin bebas sehingga dapat langsung diintegrasikan oleh client frontend.	**TERVERIFIKASI**
Koleksi Postman	Koleksi JSON lengkap tersedia pada path postman/BTNG_Showroom_Mobil_API.postman_collection.json.	**TERVERIFIKASI**

Ringkasan Submission Proyek Tugas Akhir:
- **Format Repositori:** BTNG-Backend-ArkaanTaqiyyuddin (Public)
- **Tautan GitHub:** https://github.com/Arrsiixx7/BTNG-Backend-ArkaanTaqiyyuddin
- **Tautan Live Documentation:** https://btng-backend-arkaantaqiyyuddin.cleverapps.io/docs/api
- **Batas Waktu Pengumpulan:** 28 Oktober 2026, Pukul 23.59 WIB melalui Portal Resmi BTNG UKM DNCC.